

C 프로그래밍

구조체 (Structure)

구조체란?

여러 타입의 데이터를 하나의 이름으로 묶는 사용자 정의 타입이다.

```
1 // ❌ 관련 데이터가 흩어져 있음
2 char name[32];
3 int age;
4 double gpa;
5
6 // ✅ 구조체로 묶으면 하나의 단위로 다룰 수 있음
7 struct Student {
8     char name[32];
9     int age;
10    double gpa;
11 };
```

구조체를 쓰면 관련 데이터를 하나의 타입으로 추상화할 수 있어, 함수에 전달하거나 배열로 만들기가 쉬워진다.

구조체 선언과 초기화

선언

```
struct Point {  
    int x;  
    int y;  
};
```

- `struct` 키워드 + 태그 이름으로 타입 정의
- 멤버(member)는 중괄호 안에 나열
- 세미콜론으로 끝남

초기화

```
// 순서대로 초기화  
struct Point p1 = {3, 4};  
  
// 지정 초기화 (C99)  
struct Point p2 = {.x = 10, .y = 20};
```

예제

```
1  #include <stdio.h>  
2  
3  struct Point {  
4      int x;  
5      int y;  
6  };  
7  
8  int main() {  
9      // 순서대로 초기화  
10     struct Point p1 = {3, 4};  
11  
12     // 지정 초기화 (C99)  
13     struct Point p2 = {.x = 10, .y = 20};  
14  
15     printf("p1: (%d, %d)\n", p1.x, p1.y);  
16     printf("p2: (%d, %d)\n", p2.x, p2.y);  
17  
18     // 멤버 접근 및 수정  
19     p1.x = 100;  
20     printf("p1 수정 후: (%d, %d)\n", p1.x, p1.y);  
21  
22     return 0;  
23 }
```

typedef — 타입 이름 단순화

typedef 를 쓰면 struct 키워드를 반복하지 않아도 된다.

typedef 없이

```
struct Student {  
    char name[32];  
    int age;  
};  
  
struct Student s;           // struct 필요  
struct Student arr[10];
```

typedef 사용

```
typedef struct {  
    char name[32];  
    int age;  
} Student;  
  
Student s;                   // struct 불필요  
Student arr[10];
```

멤버 접근 — . 연산자

```
Student s;  
  
// 멤버에 값 쓰기  
s.age = 20;  
  
// 문자열은 strcpy 사용  
#include <string.h>  
strcpy(s.name, "홍길동");  
  
// 멤버 읽기  
printf("%s, %d세\n", s.name, s.age);
```

변수로 선언된 구조체는 . 연산자로 멤버에 접근한다.

구조체 포인터와 -> 연산자

포인터를 통해 구조체 멤버에 접근할 때는 -> 연산자를 사용한다.

. VS ->

```
Student s;  
Student* p = &s;  
  
// 아래 두 표현은 동일하다  
p->age = 20;           // -> 연산자 (권장)  
(*p).age = 20;        // 역참조 후 .
```

함수에 구조체 전달

```
// 값 전달 - 복사본이 생성됨 (원본 불변)  
void print(Student s) { ... }  
  
// 포인터 전달 - 원본을 직접 수정 가능  
void birthday(Student* s) {  
    s->age += 1;  
}
```

예제

```
1  #include <stdio.h>  
2  
3  typedef struct {  
4      char name[32];  
5      int age;  
6  } Student;  
7  
8  void print_student(const Student* s) {  
9      // -> 연산자: 포인터를 통한 멤버 접근  
10     printf("이름: %s, 나이: %d\n", s->name, s->age);  
11 }  
12  
13 void birthday(Student* s) {  
14     s->age += 1; // 원본 수정  
15 }  
16  
17 int main() {  
18     Student s = {"홍길동", 20};  
19     print_student(&s);  
20     birthday(&s);  
21     print_student(&s);  
22     return 0;  
23 }
```

구조체와 함수

값 전달 (복사)

```
typedef struct { int x; int y; } Point;

// 구조체 전체가 복사됨
void print_point(Point p) {
    printf("(%d, %d)\n", p.x, p.y);
}

// 반환도 가능
Point make_point(int x, int y) {
    Point p = {x, y};
    return p;
}
```

- 원본이 변경되지 않음
- 구조체가 크면 복사 비용 발생

포인터 전달 (권장)

```
// const: 읽기 전용임을 명시
void print_point(const Point* p) {
    printf("(%d, %d)\n", p->x, p->y);
}

// 수정이 필요하면 const 생략
void move(Point* p, int dx, int dy) {
    p->x += dx;
    p->y += dy;
}
```

- 복사 없이 원본에 직접 접근
- `const` 로 의도를 명확히 표현

구조체 배열과 동적 할당

구조체 배열

```
Student class[30];           // 30명 분량

for (int i = 0; i < 30; i++) {
    class[i].age = 20 + i;
}
```

동적 할당

```
int n = 3;
Student* students =
    (Student*)malloc(n * sizeof(Student));

if (students == NULL) return 1;

students[0].age = 20;    // 배열처럼 접근
(students + 1)->age = 21; // 포인터 연산

free(students);
```

예제

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  typedef struct {
6      char name[32];
7      int score;
8  } Student;
9
10 int main() {
11     int n = 3;
12     Student* students = (Student*)malloc(n * sizeof(Student));
13     if (students == NULL) return 1;
14
15     strcpy(students[0].name, "홍길동"); students[0].score = 90;
16     strcpy(students[1].name, "이순신"); students[1].score = 85;
17     strcpy(students[2].name, "강감찬"); students[2].score = 92;
18
19     for (int i = 0; i < n; i++) {
20         printf("%s: %d점\n", students[i].name, students[i].score);
21     }
22
23     free(students);
24     return 0;
25 }
```

요약

```
#include <string.h> // strcpy
#include <stdlib.h> // malloc, free
```

주제

핵심

선언

```
struct 태그 { 멤버; };
```

typedef

```
struct 반복 없이 타입명만으로 사용
```

초기화

```
= {값, ...} 또는 지정 초기화 {.멤버 = 값}
```

멤버 접근

변수 → `.` 연산자, 포인터 → `->` 연산자

함수 전달

읽기만 할 때 `const Student*`, 수정 시 `Student*`

동적 할당

```
malloc(n * sizeof(Student))
```

구조체는 관련 데이터를 하나의 타입으로 묶는 C의 핵심 추상화 도구다. C++의 클래스(class)는 구조체에서 발전한 개념이다.